CSC3002
C/C++ Programming

Review – Part II
Lectures 16-21

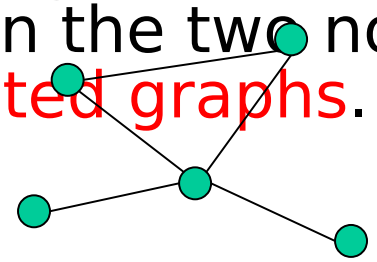
Prof. Dr. Ruizhi Chen ， SDS

Office: TxC 717

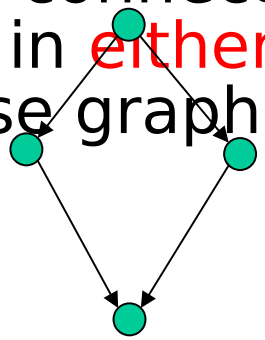
E-mail: chenruizhi@cuhk.edu.cn

The Definition of a Graph

- A graph consists of a set of nodes together with a set of arcs. The nodes correspond to the dots or circles in a graph diagram (which might be cities, words, neurons, or who knows what) and the arcs correspond to the links that connect two nodes.
- In some graphs, arcs are shown with an arrow that indicates the direction in which two nodes are linked. Such graphs are called directed graphs.
- Other graphs, arcs are simple connections indicating that one can move in either direction between the two nodes. These graphs are undirected graphs.



Undirected Graph



Directed Graph

Graph Terminology

- Mathematicians often use the words vertex and edge for the structures that computer scientists call node and arc. We'll stick with the computer science names, but you might see their mathematical counterparts in algorithmic descriptions.
- The nodes to which a particular node is connected are called its neighbors.
- In an undirected graph, the number of connections from a node is called its degree. In a directed graph, the number of arcs leading outward is called the out-degree, and the number of arcs leading inward is called the in-degree.
- A series of arcs that connect two nodes is called a path. A path that returns to its starting point is called a cycle. A simple path/cycle is a path/cycle without duplicated nodes.
- A graph in which there is a path between every pair of nodes is said to be strongly connected. A directed

Graph Representations

- Like most abstract structures, graphs can be implemented in different ways. The primary feature that differentiates these implementations is the strategy used to represent connections between nodes. In practice, the most common strategies are:
  - Storing the connections for each node in an adjacency list.
  - Storing the connections for the entire graph in an adjacency matrix.
  - Storing the connections for each node as a set of arcs.

Graph Representations

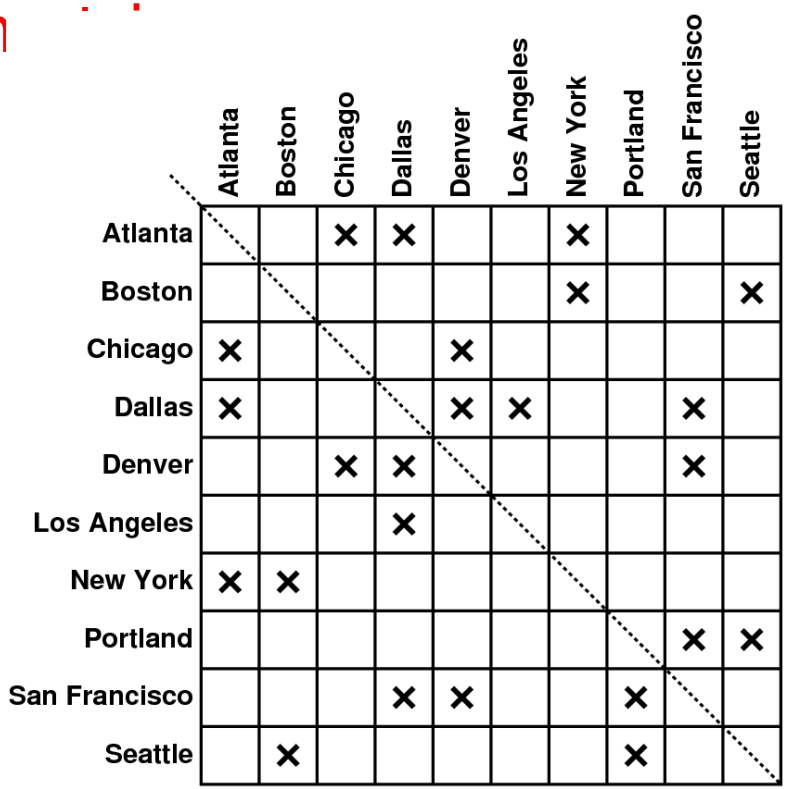
- Storing the connections for each node in an adjacency list

  Atlanta → (Chicago, Dallas, New York)
  Boston → (New York, Seattle)
  Chicago → (Atlanta, Denver)
  Dallas → (Atlanta, Denver, Los Angeles)
  Denver → (Chicago, Dallas, San Francisco)
  Los Angeles → (Dallas)
  New York → (Atlanta, Boston)
  Portland → (San Francisco, Seattle)
  San Francisco → (Dallas, Denver, Portland)
  Seattle → (Boston, Portland)

- Although simple and convenient, this representation can be inefficient when searching through the list of arcs associated with a node. This cost, O(D), where D represents the degree of the originating node, becomes more significant in

Graph Representations

- Storing the connections for the entire graph in an adjacency m



	Atlanta	Boston	Chicago	Dallas	Denver	Los Angeles	New York	Portland	San Francisco	Seattle
Atlanta			×	×			×			
Boston							×			×
Chicago	×				×					
Dallas	×				×	×			×	
Denver			×	×					×	
Los Angeles				×						
New York	×	×								
Portland									×	×
San Francisco				×	×			×		
Seattle		×						×		

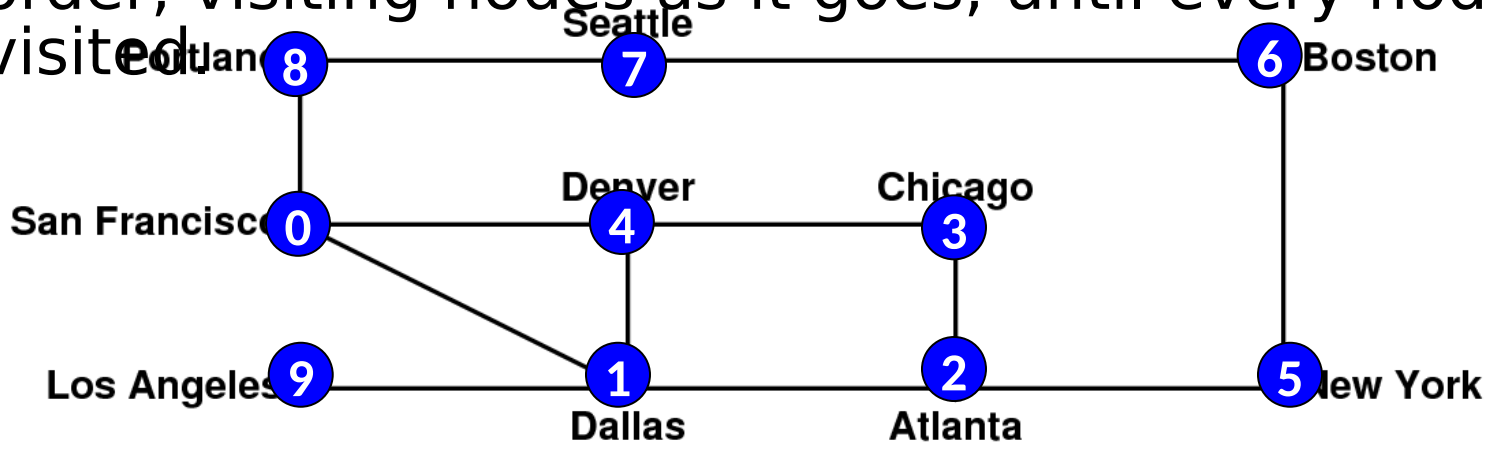
- Similar to the idea of lookup table, this representation reduces the cost of checking for connections to O(1). But the O(N^2) storage space is inefficient for a sparse graph (i.e., D << N)

Three Strategies for Graph Abstraction

1. Use low-level structures. This design uses the structure types Node and Arc to represent the components of a graph. This model gives clients complete freedom to extend these structures but offers no support for graph operations.
2. Adopt a hybrid strategy. This design defines a Graph class as a parameterized class using templates so that it can use any structures or objects as the node and arc types. This strategy retains the flexibility of the low-level model and avoids the complexity associated with the class-based approach.
3. Define classes for each of the component types. This design uses the Node and Arc classes to define the structure. In this model, clients define subclasses of the supplied types to particularize the graph data structure to their own application

Depth-First Search

- The traversal strategy of depth-first search (or DFS for short) recursively processes the graph, following each branch, assuming that the neighboring nodes are stored in alphabetical order, visiting nodes as it goes, until every node is visited.



Depth-First Search Using Stack

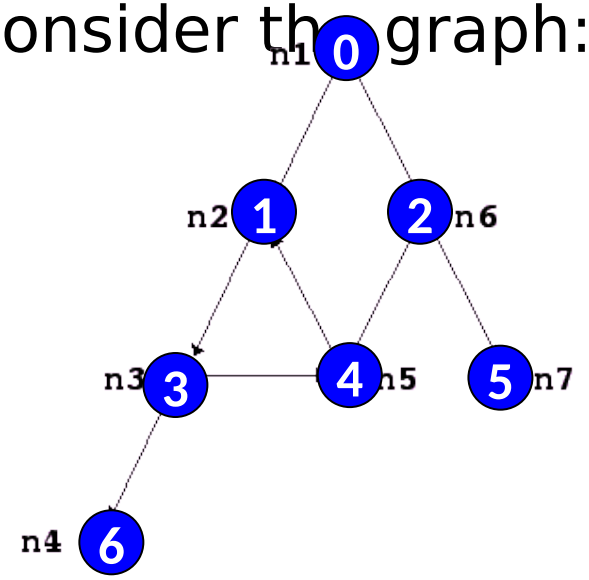
```
/* ---------- Iterative DFS ----------
g : adjacency list (0-based vertex ids)
start : source vertex
visited: boolean vector (must be sized |V|)
---------------------------------------- */
void dfs_iterative(const vector<vector<int>> &g, int start, vector<bool>&visited)
{
    stack<int> st;
    st.push(start);
    cout << "DFS: ";
    while (!st.empty())
    {
        int u = st.top();
        st.pop();

        if (visited[u])
        continue; // skip if already done
        visited[u] = true; // mark on FIRST real visit
        cout << u << ' '; // *** process vertex here ***

        // push the neibours to stack in reverse order
        for (auto it = g[u].rbegin(); it != g[u].rend(); ++it)
        if (!visited[*it]) st.push(*it);
    }
    cout << endl;
    }
```

Breadth-First Search

- The traversal strategy of breadth-first search (or BFS for short) proceeds outward from the starting node, visiting the start node, then all nodes one hop away, and so on.
- For example, consider the graph:



- Breadth-first search begins at the start node (n1), then does the one-hops (n2 and n6), then the two hops (n3, n5, and n7) and finally the three hops

Breadth-First Search Using Queue

```
void bfs_iterative(const vector<vector<int>> &g, int start, vector<bool> &visited)
{
    queue<int> q;
    q.push(start);

    while (!q.empty())
    {
        int u = q.front();
        q.pop();

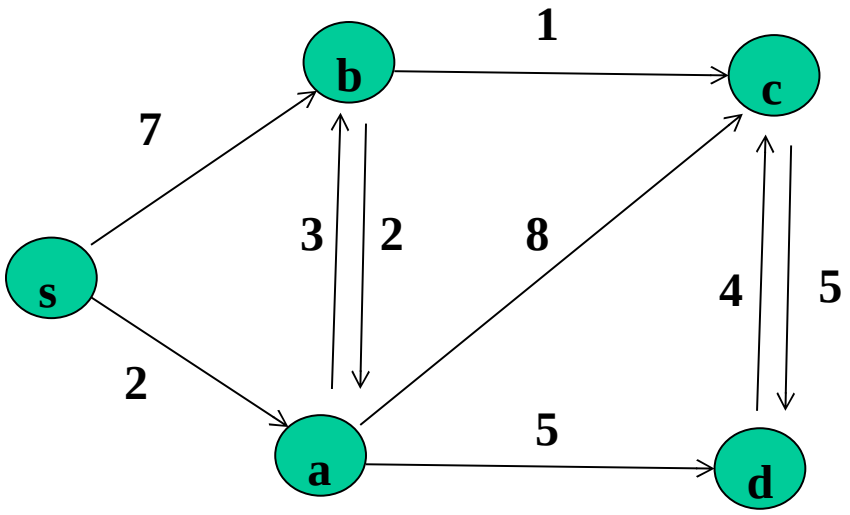
        if (visited[u]) continue; // skip if already done
        visited[u] = true; // mark on FIRST real visit
        cout << u << ' '; // *** process vertex here ***

        for (int i : g[u])
        if (!visited[i]) q.push(i);
    }
    cout << endl;
    }
```

Finding Shortest Paths: Dijkstra's Algorithm

- In general, the number of paths between two nodes in a graph grows in an exponential fashion, which means that the running time of the explore-all-paths approach is $O(2^N)$.
- One of the most useful algorithms for computing the shortest paths in a graph was developed by Edsger W. Dijkstra in 1959.
- The strategy is similar to the breadth-first search with the following additions:
  - The queue used to hold the paths delivers items in increasing order of total cost rather than in the traditional first-in/first-out order. Such queues are called priority queues.
  - The algorithm keeps track of all nodes to which the total distance has already been fixed. Distances are fixed whenever you dequeue a

The Dijkstra's Algorithm



Important Hints：

- If <a, b, c, d> is a shortest path, its sub-path <a, b, c> is also a shortest path.
- Estimate the shortest distances for all nodes
- Book keeping the previous node in the path from which the shortest distance was obtained.

- Initiate the process from the starting node,

1 . Estimate the distances (to starting point) for all going arcs,
2. update the distances for those nodes if smaller distances were obtained from step 1.
3. add all these nodes with their distances to the priority-queue
4. pop the node with "shortest distance to the starting point", repeat from step 1, until the reach of the destination.

Recursion

- One of the most important “Great Ideas” in programming is the concept of ***recursion***, which is the process of solving a problem by dividing it into smaller sub-problems of ***the same form***. The emphasized phrase is the essential characteristic of recursion; without it, all you have is a description of top-down design/stepwise refinement we have studied earlier.
- The fact that recursive decomposition generates sub-problems that have the same form as the original problem means that recursive programs will use the same function or method to solve sub-problems at different levels of the solution. In terms of the structure of the code, the defining characteristic of recursion is having ***functions that call themselves***, directly or indirectly, as the decomposition process proceeds.

The Recursive Paradigm

- Most recursive methods you encounter in an introductory course have bodies that fit the following general pattern:

```
if (test for a simple case) {
    Compute and return the simple solution without using recursion.
} else {
    Divide the problem into one or more sub-problems that have the same form.
    Solve each of the sub-problems by calling this method recursively.
    Return the solution by reassembling the results of the various sub-problems.
}
```

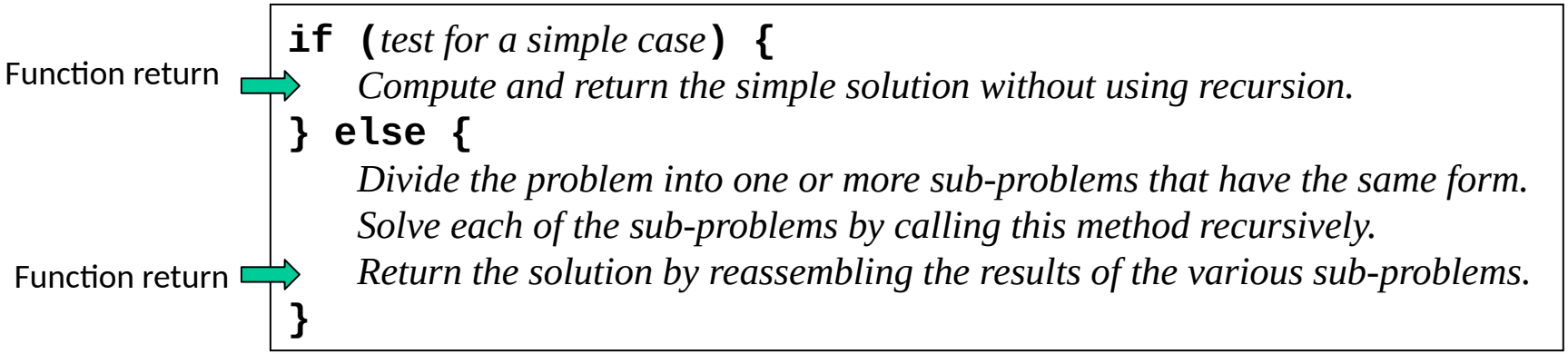
- Finding a recursive solution is mostly a matter of figuring out how to break it down so that it fits the paradigm. When you do so, you must do two things:

1. Identify ***simple cases*** that can be solved without recursion.
2. Find a ***recursive decomposition*** that breaks each instance of the problem into simpler sub-problems of the same type, which you can then solve by applying the method recursively.

The Recursive Paradigm

Calling the same function recursively ,

1) when calling a function, the operating system will copy this function (code + parameters) to stack frame
2) due to the recursive characteristics, the stack frame may have multiple copies of calls of the same recursive function.
3) when the function is executed, it will be removed from stack frame and return the value back to the caller.



The Pascal‘s Triangle

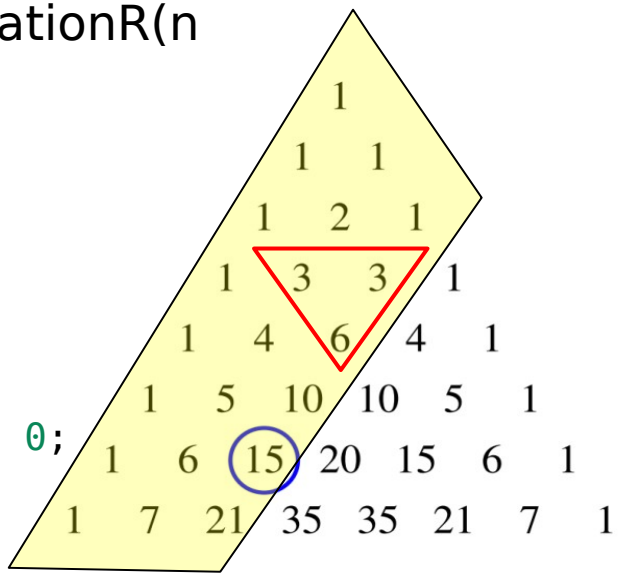
Recursive

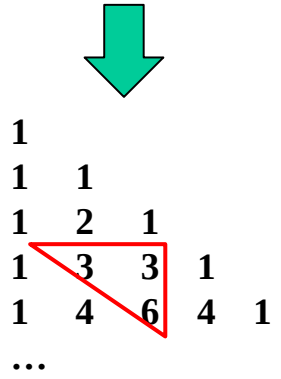
```
unsigned long CombinationR(int n, int k) {
    if (k==0|| k==n)  return 1L;
    return CombinationR(n - 1, k - 1) + CombinationR(n
- 1, k);
}
```

Iterative

```
unsigned long CombinationI(int n, int k)
{
    unsigned long a[n + 1][k + 1];
    for (int i = 0; i <= n; i++) {
            for (int j = 0; j <= k; j++) a[i][j] = 0;
            a[i][0] = a[i][i] = 1L;
    }

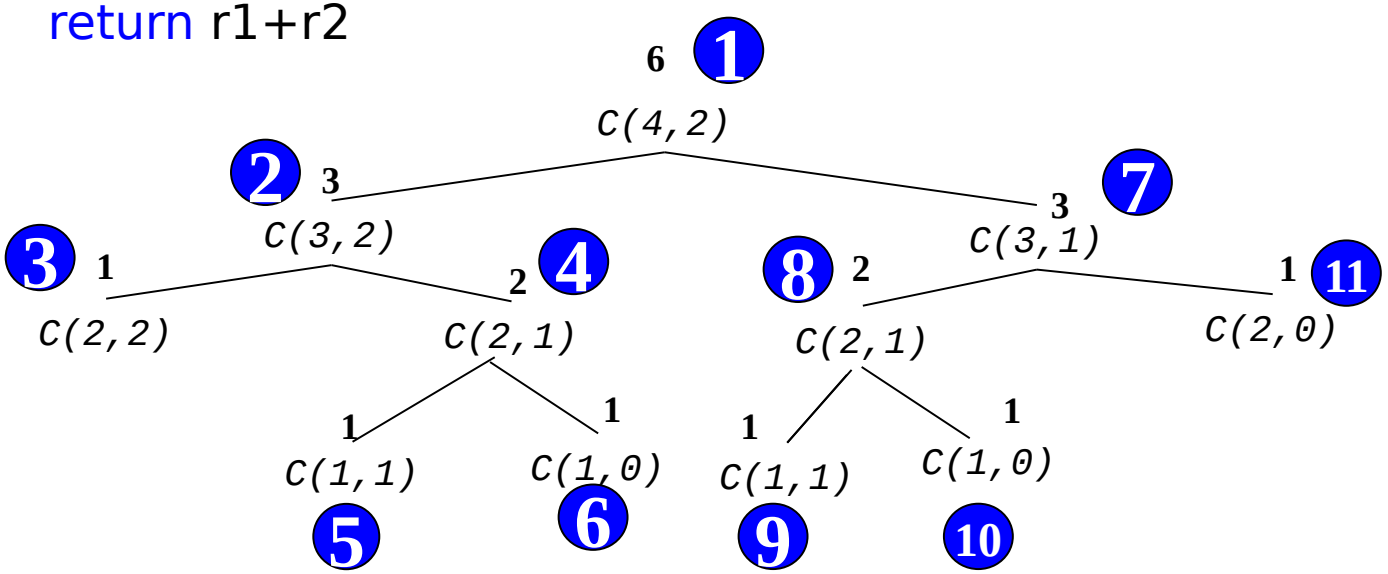
    for (int i = 2; i <= n; i++) {
            for (int j = 1; j <= k; j++) {
            a[i][j] = a[i - 1][j - 1] + a[i - 1][j];
            }
    }
    return a[n][k];
    }
```





The Pascal‘s Triangle

```
unsigned long C(int n, int k) {
    if (k==0|| k==n)  return 1L;
    unsigned long r1 = C(n - 1,
k);
    unsigned long r2 = C(n - 1,
 k - 1) ;
    return r1+r2
}
```



Preorder Root-Left-Right

Recursive Formulation of `fib(n)`

- The nth term in the Fibonacci sequence has the following recursive definition:

$$\texttt{fib(n)} = \begin{cases} \texttt{n} & \text{if } \texttt{n} \text{ is 0 or 1} \\ \texttt{fib(n - 1) + fib(n - 2)} & \text{otherwise} \end{cases}$$

- As with factorial, the recursive definition leads directly to an implementation, as follows:

```
int fib(int n) {
   if (n < 2) return n;
   return fib(n – 1) + fib(n - 2);
}
```

- Is this a good program?

  Extremely inefficient! Why?

Additive Sequence

$$t_n = t_{n-1} + t_{n-2}$$

Depending on how you choose the first two terms, you can generate many different sequences. The traditional Fibonacci sequence

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, . . .

comes from defining $t_0 = 0$ and $t_1 = 1$. If, for example, you defined $t_0 = 3$ and $t_1 = 7$, you would get this sequence instead:

3, 7, 10, 17, 27, 44, 71, 115, 186, 301, 487, 788, 1275, . . .

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
3	7	10	17	27	44	71	115	186	301

$t_9=301\ (t_0=3, t_1=7)$

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
7	10	17	27	44	71	115	186	301

$t_8=301\ (t_0=7, t_1=10)$



$t_n\ (t_0, t1) = t_{n-1}(t_1, t_0+t_1)$

Additive Sequence

```
return additiveSequence(n - 1, t0, t1) + additiveSequence(n - 2, t0,
t1);
```

```
int fib(int n) {
   return additiveSequence(n, 0, 1);
}
int additiveSequence(int n, int t0, int t1) {
   if (n == 0) return t0;
   if (n == 1) return t1;
   return additiveSequence(n - 1, t1, t0 + t1);
}
```

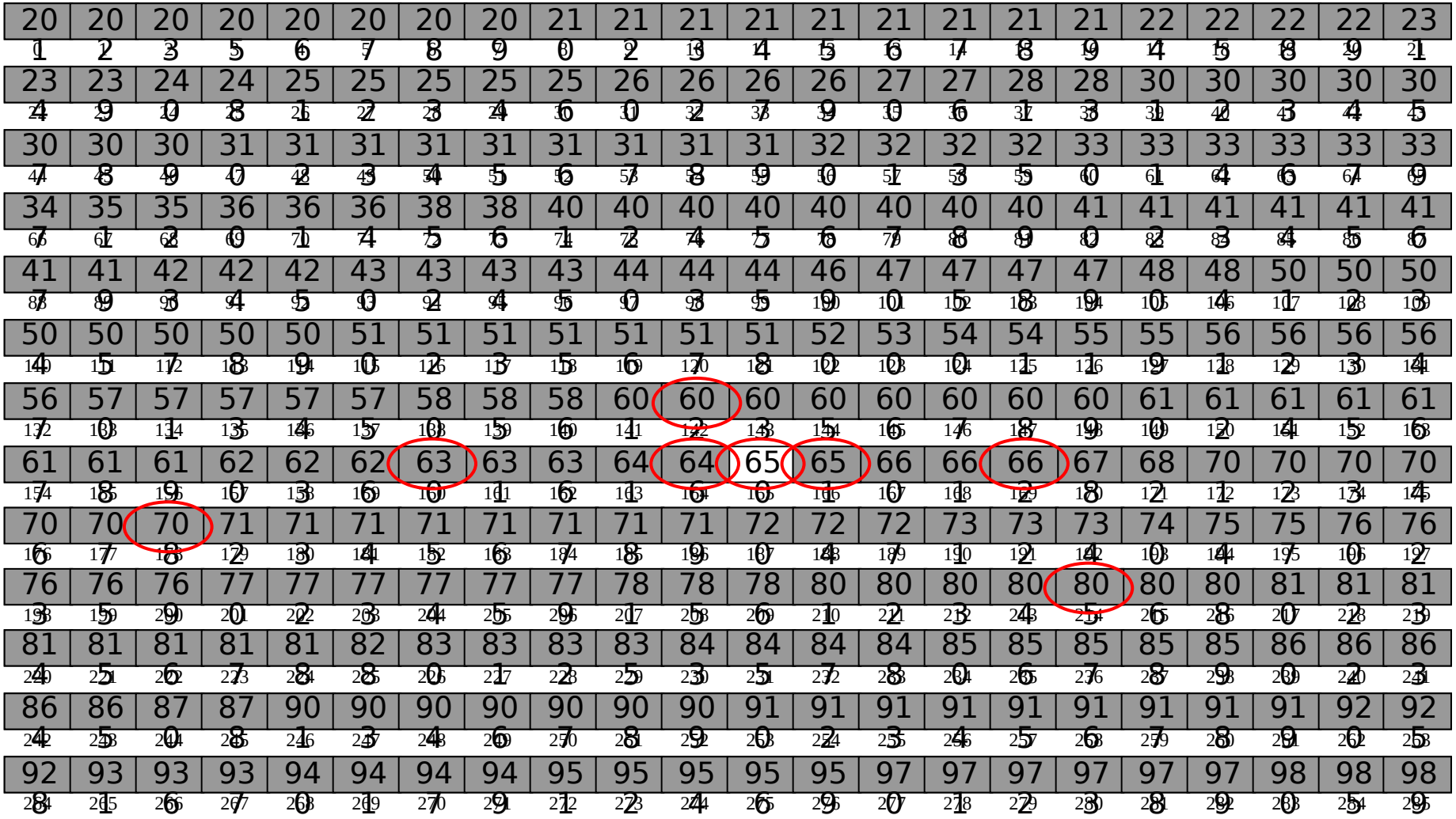
```
fib(5)
= additiveSequence(5,0,1)
= additiveSequence(4,1,1)
= additiveSequence(3,1,2)
= additiveSequence(2,2,3)
= additiveSequence(1,3,5)
= 5
```

The Idea of Binary Search

- The fact that the area code array is in ascending order makes it possible to find a particular value much more efficiently.
- The key insight is that you get more information by starting at the middle element than you do by starting at the beginning.
- When you look at the middle element in relation to the value you're searching for, there are three possibilities:
  - If the value you are searching for is greater than the middle element, you can discount every element in the first half of the array.
  - If the value you are searching for is less than the middle element, you can discount every element in the second half of the array.
  - If the value you are searching for is equal to the middle element, you can stop because you've found the value you're looking for.
- You can repeat this process on the elements that remain after each cycle. Because this algorithm proceeds by dividing the list in half each time, it is called ***binary search***.

Binary Search (Area Code Example)

Binary search needs to look at only eight elements to find 650.



Mutual Recursion

- In some cases, recursive processes involve more than a single function. Instead, one function calls another, which calls another, which eventually circles back around to the original function. Such situations are called ***mutual recursion***.
- The text uses the following simple example to illustrate mutual recursion, although it is inefficient and seems unnecessary!

```
bool isOdd(unsigned int n) {
   return !isEven(n);
}

bool isEven(unsigned int n) {
   if (n == 0) return true;
   return isOdd(n – 1);
}
```

The Tower of Hanoi Solution

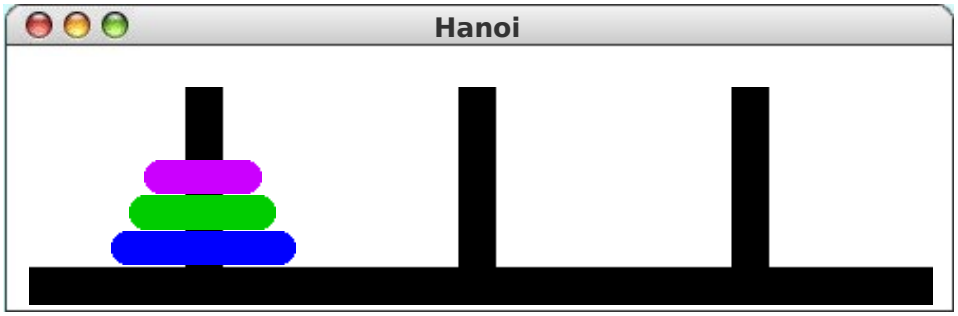
- If you play the game for a few moments, it becomes clear that you can solve the problem by dividing it into these three steps:
  1. Move the entire stack consisting of the top n-1 disks from spire A to spire C (with the help of B).
  2. Move the bottom disk from spire A to spire B.
  3. Move the stack of n-1 disks from spire C to spire B (with the help of A).

Moving n disks from A to B

The Towers of Hanoi Solution

```
int main() {
void moveTower(int n, char start, char finish, char temp) {
void moveTower(int n, char start, char finish, char temp) {
void moveTower(int n, char start, char finish, char temp) {
  if (n == 1) {
     moveSingleDisk(start, finish);
  } else {
     moveTower(n - 1, start, temp, finish);
     moveSingleDisk(start, finish);
     moveTower(n - 1, temp, finish, start);
  }
}
```

n	start	finish	temp
1	'A'	'B'	'C'



skip simulation

The Subset-Sum Problem

- The ***subset-sum problem*** can be expressed as follows:

> *Given a set of integers and a target value, determine whether it is possible to find a subset of those integers whose sum is equal to the specified target.*

- For example, given the set { –2, 1, 3, 8 } and the target value 7, the answer to the subset-sum question is?
- Yes, because the subset { –2, 1, 8 } adds up to 7.
- What if the target value is 5?
- The answer would be no.
- This problem is a little more difficult than the previous subset problem because there is an extra target value in play.

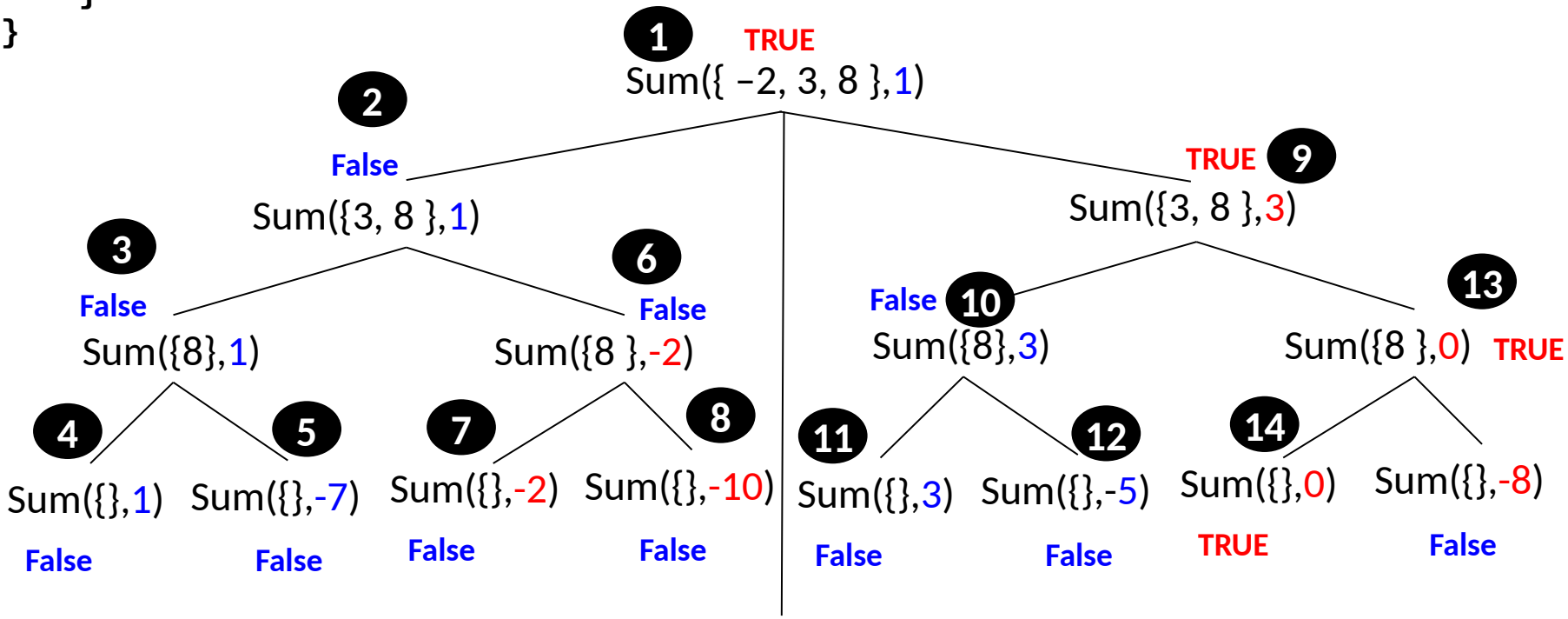
The Subset-Sum Problem

- The recursive solution to this problem can again follow the ***inclusion-exclusion pattern***.
- The key insight you need to solve this problem is that there are two ways that you might be able to produce the desired target sum after you have identified a particular element.
  - One possibility is that the subset you're looking for *excludes* that element, in which case it must be possible to generate the value **target** using only the rest of the set of elements.
  - The other possibility is that the subset you're looking for *includes* that element. For that to happen, it must be possible to take the rest of the set and produce the value **target-element.**

Code for the Subset-Sum Program

```
/*
 * Function: subsetSumExists
 * Usage: if (subsetSumExists(set, target) . . .
 * ----------------------------------------
 * Determines whether it is possible to choose elements from set so that
 * the sum is equal to target.  The recursion uses the inclusion-exclusion
 * pattern.  This pattern begins by choosing a particular element (this
 * code chooses the first element) and then continues by checking whether
 * it is possible to create the target without using that element or
 * whether it is possible to include that element and then find a subset
 * of the rest that adds up to the target after subtracting the element.
 */
bool subsetSumExists(set<int>& Set, int target) {
    if (Set.empty()) {
        return target == 0;
    }
    else {
        int element = *Set.begin(); //get the first element
        set<int> rest = Set; //copy the Set
        rest.erase(rest.begin());//remove the first element, to get the rest
        return subsetSumExists(rest, target)
            || subsetSumExists(rest, target - element);
    }
}
```

```
bool subsetSumExists(set<int>& Set, int target) {
    if (Set.empty()) {
        return target == 0;
    }
    else {
        int element = *Set.begin(); //get the first element
        set<int> rest = Set; //copy the Set
        rest.erase(rest.begin());//remove the first element, to get the rest
        return subsetSumExists(rest, target)
            || subsetSumExists(rest, target - element);
    }
}
```



If you find at least one subset is TRUE , the final result is TRUE

Analysis of Algorithms

- **When we analyze algorithms, we should employ mathematical techniques that analyze algorithms independently of *specific implementations, computers, or data*.**

- **To analyze algorithms:**
  - **First, we start to count the number of significant operations in a particular solution to assess its efficiency.**
  - **Then, we will express the efficiency of algorithms using growth functions.**

General Rules for Estimation

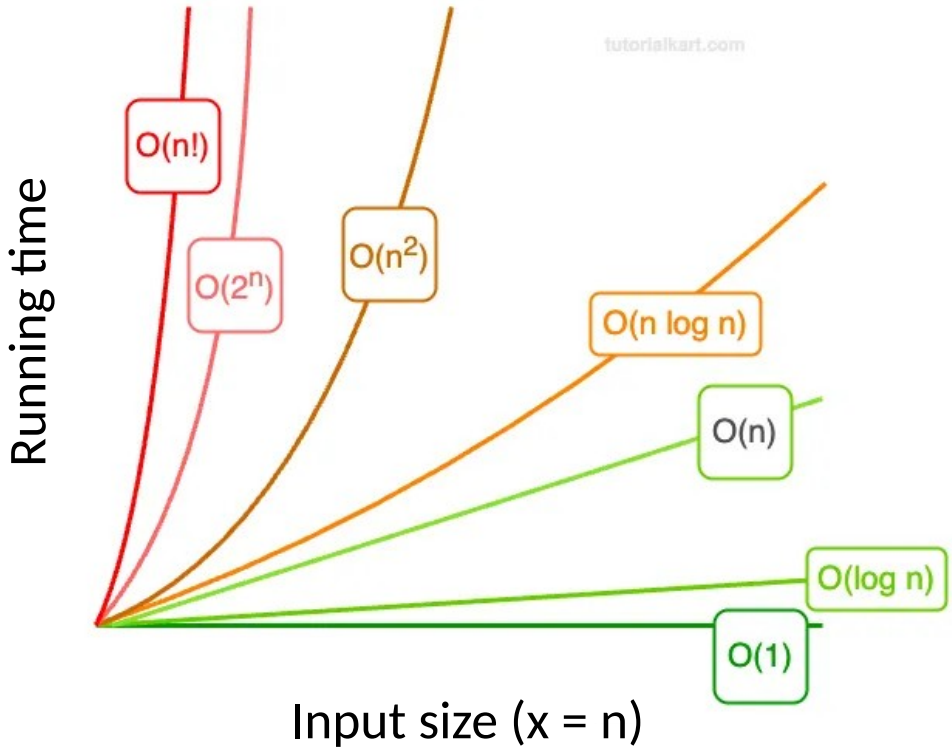
- **Loops: The running time of a loop is at most the running time of the statements inside of that loop times the number of iterations.**
- **Nested Loops: Running time of a nested loop containing a statement in the inner most loop is the running time of statement multiplied by the product of the sized of all loops.**
- **Consecutive Statements: Just add the running times of those consecutive statements.**
- **If/Else: Never more than the running time of the test plus the larger of running times of S1 and S2.**

Algorithm Growth Rates

- We measure an algorithm's time requirement as a function of the *problem size*.
  - Problem size depends on the application: e.g. number of elements in a list for a sorting algorithm, the number users for a social network search.
- So, for instance, we say that (if the problem size is n)
  - Algorithm A requires $5*n^2$ time units to solve a problem of size n.
  - Algorithm B requires $7*n$ time units to solve a problem of size n.
- The most important thing to learn is how quickly the algorithm's time requirement grows as a function of the problem size.
  - Algorithm A requires time proportional to n^2.
  - Algorithm B requires time proportional to n.
- An algorithm's proportional time requirement is known as *growth rate*.
- We can compare the efficiency of two algorithms by comparing their growth rates.

Common Growth Rates

Function	Growth Rate Name
c	Constant
log N	Logarithmic
$\log^2 N$	Log-squared
N	Linear
N log N	Log-linear
N^2	Quadratic
N^3	Cubic
2^N	Exponential



Definition of the Order of an Algorithm

Definition:

Algorithm A is order g(n) – denoted as O(g(n)) – if constants k and n_0 exist such that A requires no more than k*g(n) time units to solve a problem of size n n_0.



Find the constants k and n_0, so that

$$f(n) \leq k*g(n) \text{ for all } n \quad n_0$$

f(n) is the function of execution time.

- The requirement of n n_0 in the definition of O(g(n)) formalizes the notion of sufficiently large problems.
  - In general, many values of k and n_0 can satisfy this definition.

Order of an Algorithm

If an algorithm requires $f(n) = n^2 - 3*n + 10$ seconds to solve a problem size n. If constants k and n_0 exist such that

$$k*n^2 > n^2 - 3*n + 10 \quad \text{for all } n \quad n_0 .$$

the algorithm is order n^2 (In fact, k is 3 and n_0 is 2)

$$3*n^2 > n^2 - 3*n + 10 \quad \text{for all } n \quad 2 .$$

Thus, the algorithm requires no more than $k*n^2$ time units for $n \quad n_0$,

So it is $O(n^2)$ $(g(n) = n^2)$

Properties of Growth-Rate Functions

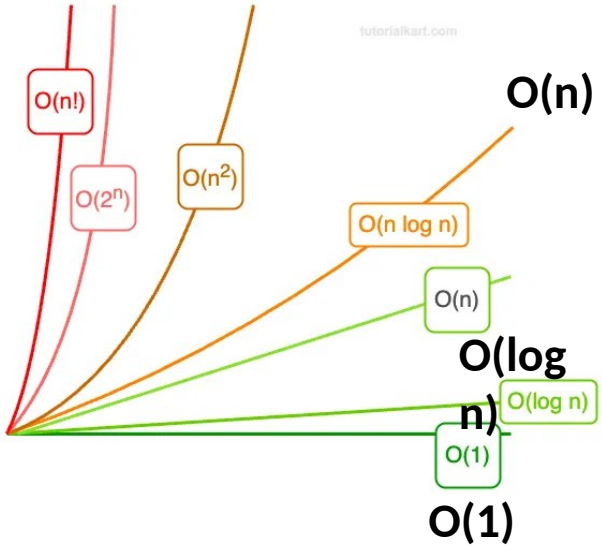
1. ***We can ignore low-order terms in an algorithm's growth-rate function.***
   - **If an algorithm is $O(n^3+4n^2+3n)$, it is also $O(n^3)$.**
   - **We only use the higher-order term as algorithm's growth-rate function.**

2. ***We can ignore a multiplicative constant in the higher-order term of an algorithm's growth-rate function.***
   - **If an algorithm is $O(5n^3)$, it is also $O(n^3)$.**

3. ***$O(f(n)) + O(g(n)) = O(f(n)+g(n))$***
   - **We can combine growth-rate functions.**
   - **If an algorithm is $O(n^3) + O(4n^2)$, it is also $O(n^3+4n^2)$ ⇒ So, it is $O(n^3)$.**
   - **Similar rules hold for multiplication.**

Performance comparison of typical data structures

Data Structure	Access	Search	Insertion	Deletion	Notes
Array	O(1)	O(n)	O(n)	O(n)	Fixed-size; resizing takes O(n)
Singly Linked List	O(n)	O(n)	O(1)	O(1)	Sequential access
Doubly Linked List	O(n)	O(n)	O(1)	O(1)	Supports two-way traversal
Hash Table	O(1)	O(1)	O(1)	O(1)	O(n) in worst case (hash collisions)
Binary Search Tree (BST)	O(log n)	O(log n)	O(log n)	O(log n)	Worst case O(n) if unbalanced
(Balanced BST)	O(log n)	O(log n)	O(log n)	O(log n)	Self-balancing BST, guarantees log n operations



Sorting Algorithms

- There are many sorting algorithms
- In this class we will learn:
  - Selection Sort
  - Insertion Sort
  - Bubble Sort
  - Merge Sort
  - Quick Sort
  - BST Tree Sort
  - Heap Sort
- These are among the most fundamental sorting algorithms

Selection Sort

- Partition the input list into a sorted and unsorted part (initially sorted part is empty)
- Select the smallest element and put it to the end of the sorted part
- Increase the size of the sorted part by one
- Repeat this $n-1$ times to sort a list of n elements

Insertion Sort

- **Insertion sort is a simple sorting algorithm that is appropriate for small inputs.**
  - **Most common sorting technique used by card players.**
- **Again, the list is divided into two parts: sorted and unsorted.**
- **In each pass, the first element of the unsorted part is picked up, transferred to the sorted sublist, and inserted at the appropriate place.**
- **A list of *n* elements will take at most *n - 1* passes to sort the data.**

Bubble Sort

Starting from left (index = 0) to right (index =n), repeatedly swap adjacent elements that are out of order, smaller element to the left, larger element to the right. With this process, bubble the largest element to the right-most slot.

Reducing the problem size from n to n-1, repeat the last step, until no swapping needed, → sorted.

Mergesort

- **Mergesort algorithm is one of two important divide-and-conquer sorting algorithms (the other one is quicksort).**
- **It is a recursive algorithm.**
  - **Divides the list into halves,**
  - **Sort each halve separately (recursively), and**
  - **Then merge the sorted halves into one sorted array.**

The Merge Sort Idea

1. **Divide the vector into two halves: `v1` and `v2`.**
2. **Sort each of `v1` and `v2` recursively.**
3. **Clear the original vector.**
4 **Merge elements into the original vector by choosing the smallest element from `v1` or `v2` on each cycle.**

vec

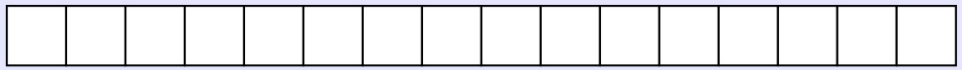
23	25	36	36	50	65	80	83	94	98

v1

v2

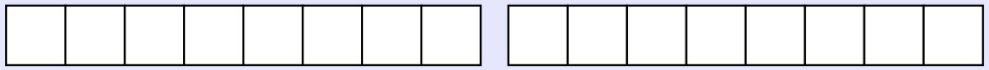
The Complexity of Merge Sort

Sorting a vector of size N



N operations

requires sorting two vectors of size N / 2



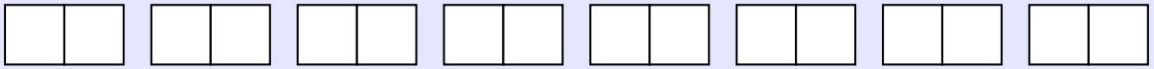
$2 \times N/2$ operations

requires sorting four vectors of size N / 4



$4 \times N/4$ operations

requires sorting eight vectors of size N / 8



$8 \times N/8$ operations

and so on.

The work done at each level (*i.e.*, the sum of the work done by all the calls at that level) is proportional to the size of the vector. The running time is therefore proportional to N times the number of levels.

How Many Levels Are There?

- **The number of levels in the merge sort decomposition is equal to the number of times you can divide the original vector in half until there is only one element remaining. In other words, what you need to find is the value of *k* that satisfies the following equation:**

$$1 = \underbrace{N \,/\, 2 \,/\, 2 \,/\, 2 \,/\, 2 \cdots / \, 2}_{k \text{ times}}$$

- **You can simplify this formula using basic mathematics:**

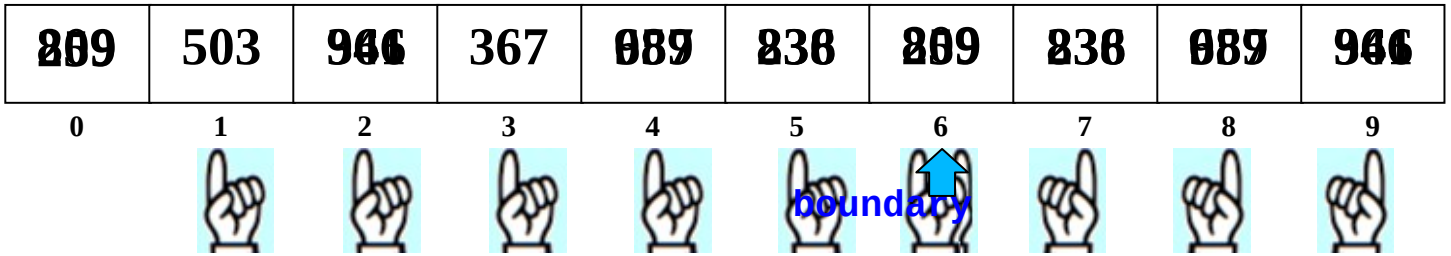
$$1 = N \,/\, 2^k \quad \Rightarrow \quad 2^k = N \quad \Rightarrow \quad k = \log_2 N$$

- **Because the number of levels *k* is $\log_2$N and the amount of work done at each level is to *N*, the total amount of work is proportional to $O(N\log_2 N)$.**

The Quicksort Algorithm

- **Most sorting libraries use some variation of the *Quicksort algorithm*, which was developed by Tony Hoare in the 1950s.**
- **The Quicksort algorithm consists of two phases:**
  1. ***Partition*. In the partition phase, the elements of the array are reordered so that the array begins with a set of "small" elements and ends with a set of "big" elements, where the distinction between "small" and "big" is made relative to an element called the *pivot,* which appears at the boundary between the two regions.**
  2. ***Sort*. In the sort phase, the Quicksort algorithm is applied recursively to the "small" and "big" subarrays, which leaves the entire array sorted.**

Partitioning the Array



1. **Select the first element as the *pivot* and set it aside.**
2. **Keep two indices into the remaining elements, starting at each end.**
3. **Advance the right index until until it is smaller than the pivot. Then advance the left index until it is larger than the pivot.**
4. **Swap the elements at the two indices.**
5. **Repeat the process until the indices coincide.**
6. **If pivot is large than the element at the coincide index, swap the pivot and the coincide index, and return the coincide index as the boundary index.**

BST Tree Sort Analysis

- Inserting an item into a binary search tree:
  - Worst Case: O(n)
  - Average Case: $O(\log_2 n)$

- Inserting *n* items into a binary search tree:
  - Worst Case: $O(n^2)$ ➔ $(1+2+\ldots+n) = O(n^2)$
  - Average Case: $O(n*\log_2 n)$

- Inorder traversal and copy items back into array ➔ O(n)
- Thus, tree sort is

➔ **$O(n^2)$ in worst case**, and

➔ **O(nlogn) in average case.**

Heapsort

- A binary heap can be used to sort *N* items by
  - 1) Inserting every item into a binary heap (O(N)) and
  - 2) Extracting every item by calling deleteMin *N* times (O(N$\log_2$N)), thus sorting the result.

Analysis of Heapsort

- It is an O(N log N) algorithm for best, average and worst cases:
  - First phase: Build heap O(N)
  - Second phase: N deleteMin (pop/peek) operations: O(NlogN).

$$O(N + N\log_2 N) = O(N\log_2 N)$$

The complexity of different sort algorithms

Sorting complexity	Best	Average	Worst
Selection sort	N^2	N^2	N^2
Insertion sort	N	N^2	N^2
Bubble sort	N	N^2	N^2
Merge sort	$N \log N$	$N \log N$	$N \log N$
Quicksort	$N \log N$	$N \log N$	N^2
HeapSort	$N \log N$	$N \log N$	$N \log N$
BST Tree Sort	$N \log N$	$N \log N$	N^2

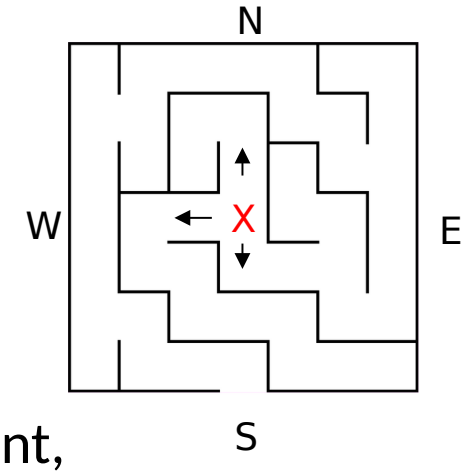
Backtracking

- For many real-world problems, the solution process consists of working your way through a sequence of decision points in which each choice leads you further along some path.

- If you reach a dead end or otherwise discover that you have made an incorrect choice somewhere along the way, you have to backtrack to a previous decision point and try a different path.

- Algorithms that use this approach are called ***backtracking algorithms***.

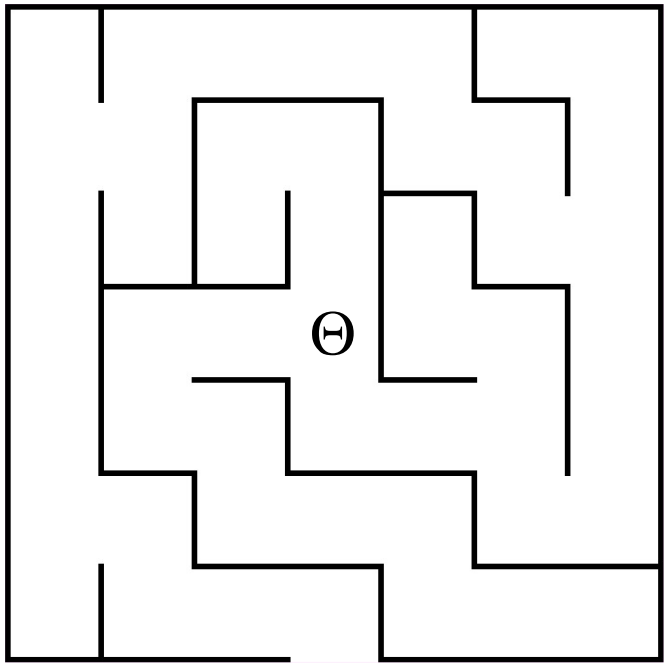
An Iterative View of Mazes



1. Create the maze
    1) Mark the location starting and ending points
    2) Mark the “wall” points of the Maze, which is not allowed to be occupied/cross
2. Starting from the Starting point as the current point,
3. Check the surrounding points of the current point, starting from, for example, North, South, West, East (can be in any order), if it is a “walkable”, put it to stack (with row, column information)
4. Having putted all “walkable” points to stack, marked the current point as a “visited” point, turn it to an unwalkable point, and will not visit it again.
5. Pop a walkable point from stack as a new current point, repeat 3-4
6. If a dead end occurs, go back to 5, until reaching the ending point.

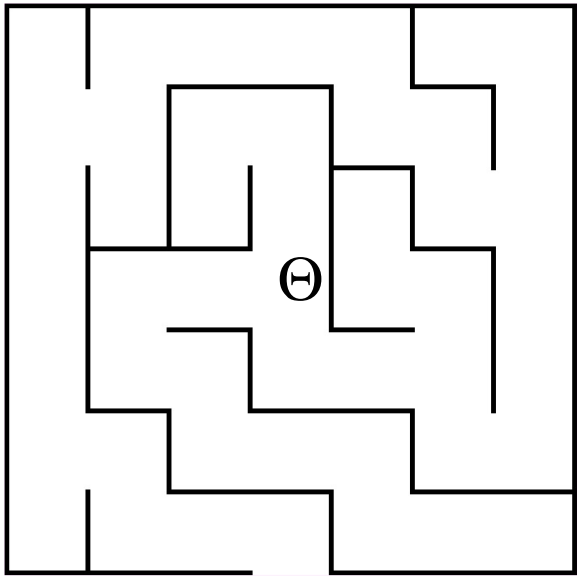
A Recursive View of Mazes

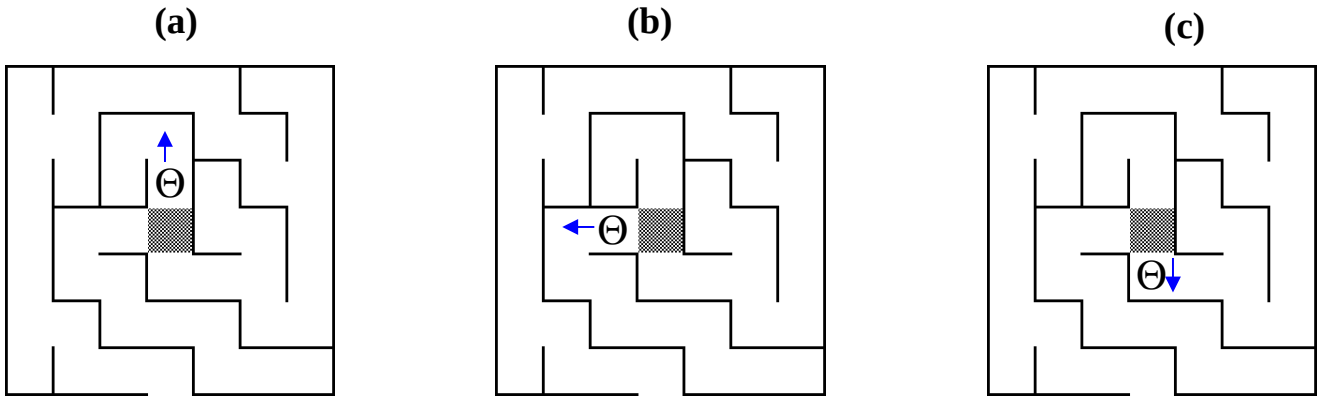
- It is also possible to solve a maze recursively. Before you can do so, however, you have to find the right recursive insight.
- Consider the maze shown at the right. How can Theseus transform the problem into one of solving a simpler maze?
- The insight you need is that a maze is solvable only if it is possible to solve one of the simpler mazes that results from shifting the starting location to an adjacent square and taking the current square out of the maze completely (one-step closer to finish)



A Recursive View of Mazes

- Thus, the original maze is solvable only if one of the three mazes at the bottom of this slide is solvable.
- Each of these mazes is "simpler" because it contains fewer squares.
- The simple cases are:
  - Theseus is outside the maze
  - There are no directions left to try

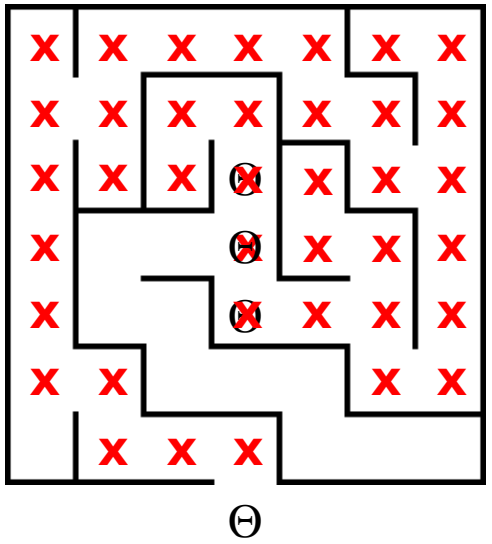




Tracing the **solveMaze** Function

```
bool solveMaze(Maze & maze, Point start) {
   if (maze.isOutside(start)) return true;
   if (maze.isMarked(start)) return false;
   maze.markSquare(start);
   for (Direction dir = NORTH; dir <= WEST; dir++) {
      if (!maze.wallExists(start, dir)) {
         if (solveMaze(maze, adjPt(start, dir))) {
            return true;
         }
      }
   }
   maze.unmarkSquare(start);
   return false;
}
```

start	dir
(3, 4)	



A Simpler Game

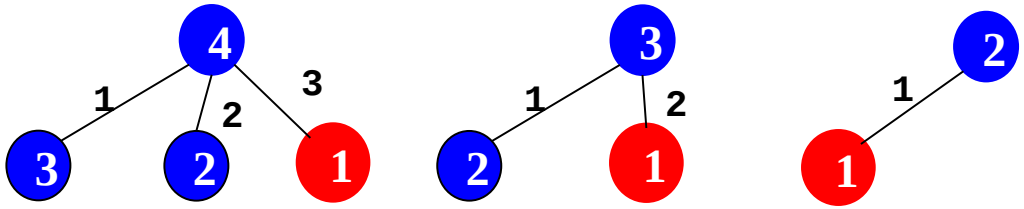
- Chess is far too complex a game to serve as a useful example. The text uses a much simpler game called ***Nim***, which is representative of a large class of two-player games.
- In Nim, the game begins with a pile of coins between two players. The starting number of coins can vary and should therefore be easy to change in the program.
- In alternating turns, each player takes one, two, or three coins from the pile in the center.
- The player who takes the last coin loses.

Good Moves and Bad Positions

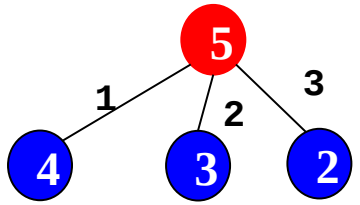
- The essential insight behind the Nim program is bound up in the following mutually recursive definitions:
  - A ***good move*** is one that leaves your opponent in a bad position.
  - A ***bad position*** is one that offers no good moves.
  - E.g., if you ever find yourself with just one coin on the table, you're in a bad position (you have to take that coin and lose). On the other hand, things look good if you find yourself with two, three, or four coins. In any of these cases, you can always take all but one of the remaining coins (good moves), leaving your opponent in the bad position of being stuck with just one coin.
- The implementation of the Nim game is really nothing more than a translation of these definitions into code.

Coding the Nim Strategy

Coins on Table	Is a bad position	Num. Takens for good move	Num of coins left for your opponent
1	Yes	Loss game	
2	No	1	1
3	No	2	1
4	No	3	1
5	Yes	NO_GOOD_MOVE	2-4 （good position for opponent）



Good positions, leave your opponent 1 coins



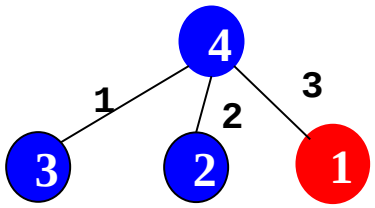
Bad position, you take any coins will leave your opponent a good position (2, or 3, or 4)

Coding the Nim Strategy

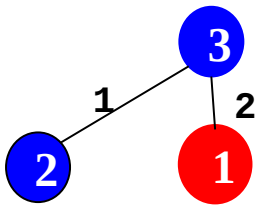
Good positions

Bad Positions

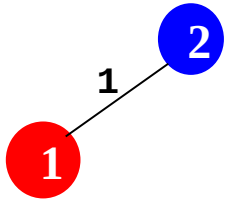
Good Move =3

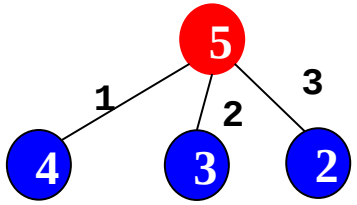


Good Move =2



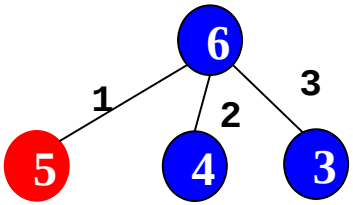
Good Move =1



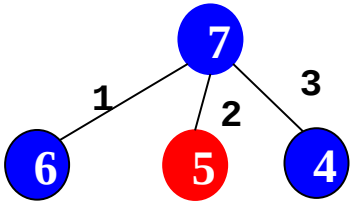


NO_GOOD_MOVE

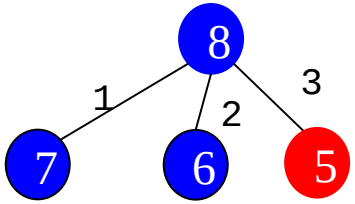
Good Move =1

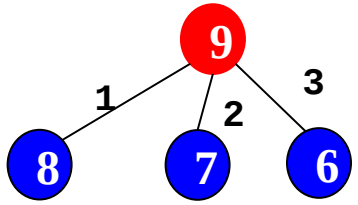


Good Move =2



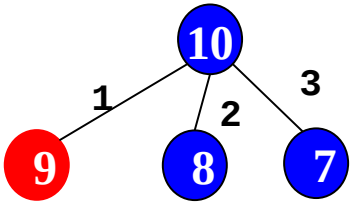
Good Move =3



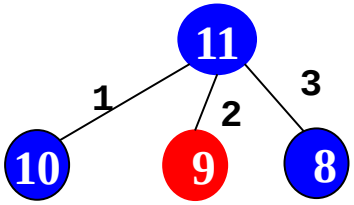


NO_GOOD_MOVE

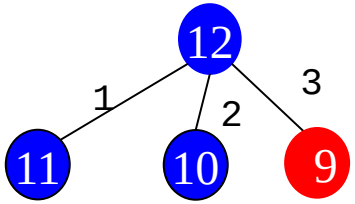
Good Move =1

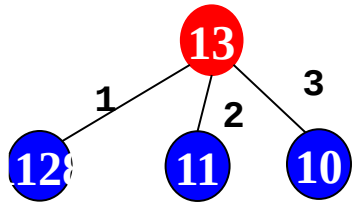


Good Move =2



Good Move =3





NO_GOOD_MOVE

Coding the Nim Strategy

```
/*
 * Looks for a winning move, given the specified number of coins.
 * If there is a winning move in that position, findGoodMove returns
 * that value; if not, the method returns the constant NO_GOOD_MOVE.
 * This implementation depends on the recursive insight that a good
move
 * is one that leaves your opponent in a bad position and a bad
position
 * is one that offers no good moves.
 */

   int findGoodMove(int nCoins) {
      int limit = (nCoins < MAX_MOVE) ? nCoins : MAX_MOVE;
      for (int nTaken = 1; nTaken <= limit; nTaken++) {
         if (isBadPosition(nCoins - nTaken)) return nTaken;
      }
      return NO_GOOD_MOVE;
   }

/*
 * Returns true if nCoins is a bad position.  Being left with a single
 * coin is clearly a bad position and represents the simple case.
 */

   bool isBadPosition(int nCoins) {
      if (nCoins == 1) return true;//or better with (nCoins%4 == 1)
      return findGoodMove(nCoins) == NO_GOOD_MOVE;
   }
```

The Minimax Algorithm

- Games like Nim are simple enough that it is possible to solve them completely in a relatively small amount of time.
- For more complex games, it is necessary to cut off the analysis at some point and then evaluate the position, presumably using some function that looks at a position and returns a ***rating*** for that position. Positive ratings are good for the player to move; negative ones are bad.
- When your game player searches the tree for best move, it can't simply choose the one with the highest rating because you control only half the play.
- What you want instead is to choose the move that minimizes the maximum rating available to your opponent. This strategy is called the ***minimax*** algorithm.
- For complex game, we cannot evaluate all possible steps, but only to a certain depth.

The Minimax Algorithm

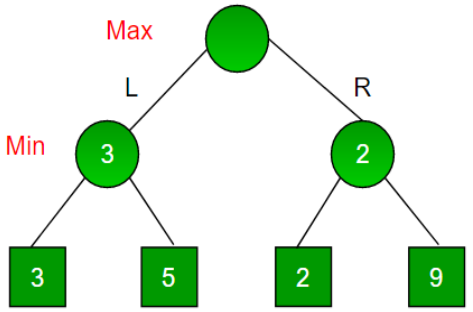
Minimax is a backtracking algorithm used in decision making and game theory to find the optimal move for a player, assuming that the opponent also plays optimally.

Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision, for example,

- *Max* goes LEFT: It is now the turn of *Min*. *Min* now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3

- *Max* goes RIGHT: It is now the turn of *Min*, *Min* now has a choice between 2 and 9. He will choose 2 as it is the least among the two values.

Being the maximizer, *Max* would choose the larger value that is 3. Hence the optimal move for the maximizer is to go LEFT and the optimal value is 3.



The Alpha-Beta Pruning Algorithm

Alpha-Beta pruning is an optimization technique for the minimax algorithm. It allows us to search much faster and even go into deeper levels in the game tree. It cuts off branches in the game tree which need not be searched because there already exists a better move available. It is called Alpha-Beta pruning because it passes 2 extra parameters in the minimax function, namely alpha and beta.

Alpha is the best value that the **maximizer** currently can guarantee at that level or above.
Beta is the best value that the **minimizer** currently can guarantee at that level or below.

When Beta <= Alpha, better move found